

Unit 1: Introductory Concepts of DBMS (10%)

Q1. What is a database system? Explain the purpose of DBMS.

Answer:

- A **database** is a collection of related data that is organized so it can be easily accessed, managed, and updated.
- A **Database Management System (DBMS)** is software that allows users to **store, retrieve, update, and manage** data efficiently.
- **Purpose of DBMS:**
 1. **Data storage & retrieval** – Provides systematic way to store large volumes of data and retrieve it quickly.
 2. **Data abstraction** – Hides low-level storage details from the user.
 3. **Data consistency** – Ensures data is accurate and up to date.
 4. **Data security** – Provides authentication and authorization to protect data.
 5. **Multi-user support** – Allows multiple users to access data at the same time without conflicts.
 6. **Backup & recovery** – Recovers data in case of failure.

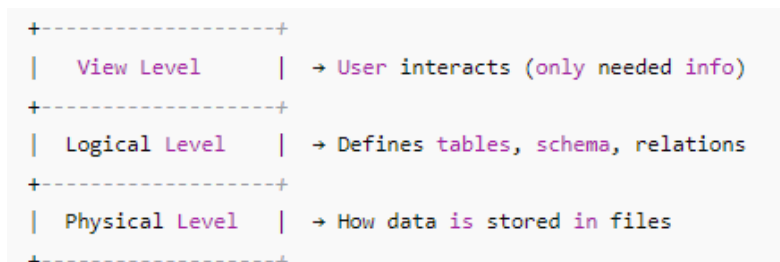
□ **In short:** A database system (DBMS) is used to store, manage, and protect data in an organized way, ensuring easy access, consistency, and security.

Q2. Explain the three levels of data abstraction with a neat diagram. (IMP)

Answer: DBMS provides **three levels of data abstraction** to hide the internal details of data storage from users and make the system easier to use.

1. **Physical Level (Lowest level)**
 - Describes **how data is actually stored** in memory or disk.
 - Focuses on storage structures like **files, indexes, blocks**.
 - Users do not see this level.
 - **Example:** Student data stored in **B+ tree files** for fast search.
2. **Logical Level (Middle level)**
 - Describes **what data is stored** and the **relationships among data**.
 - Defines **tables, attributes, relationships, and constraints**.
 - Independent of how data is physically stored.
 - **Example:** Table Student(Roll_No, Name, Branch, CGPA)
3. **View Level (Highest level)**
 - Describes **only part of the database** needed by a specific user.
 - Provides **security** by showing only required data.
 - **Example:** A teacher sees only Roll_No and Marks of students, not their phone numbers or email.

Diagram:



□ **In short:** These 3 levels hide storage details and make database use simple, secure, and efficient.

Key points:

- Hides storage details from users.
- Provides **flexibility, security, and simpler database usage.**

Q3. Define data independence. Differentiate logical vs physical data independence. (IMP)

Answer:

- **Data Independence** = Ability to **change the database structure at one level** without affecting higher levels or application programs.
- **Purpose:** Reduces maintenance cost and allows flexibility.

Types of Data Independence:

1. **Logical Data Independence**

- Changing **logical schema** (tables, attributes, relationships) **without affecting applications.**
- Harder to achieve as programs may rely on table structure.
- **Example:** Adding a column Email in Student table should not affect existing queries.

2. **Physical Data Independence**

- Changing **physical storage** (files, indexes, devices) **without affecting logical schema or programs.**
- Easier to achieve.
- **Example:** Changing from sequential files to B+ tree indexing does not affect queries.

Difference Table (Expanded)

Aspect	Logical Data Independence	Physical Data Independence
Schema Change	Logical schema (tables, attributes)	Physical schema (storage, indexing)
Impact on Programs	Application programs may need update	Application programs remain unaffected
Difficulty	Harder to achieve	Easier to achieve
Level of Abstraction	Middle level (Logical)	Lowest level (Physical)
Scope of Change	Changes in structure of tables & relationships	Changes in storage details only
Effect on Users	Users may need updated queries if not independent	Users are not affected at all
Examples	Adding/removing attributes, new table relationships	Changing file structure, indexing method

In short:

- **Logical independence:** Change tables or relationships without breaking programs.
- **Physical independence:** Change storage methods without affecting tables or queries.

Q4. List and explain the roles of different database users.**Answer:**

Different users interact with a DBMS in different ways depending on their roles and level of expertise. The main types are:

1. **Application Programmers**
 - Write application programs to interact with the database using **SQL, PL/SQL, or API calls**.
 - Responsible for developing user interfaces and application logic.
 - Example: Creating a student management system that fetches data from the Student table.
2. **End Users**
 - **Naive Users:** Use **ready-made applications** without understanding database structure.
 - Example: ATM users withdrawing money.
 - **Casual Users:** Occasionally access the database using **simple queries or reports**.
 - Example: Teacher generating student result reports.
 - **Sophisticated Users:** Can write complex queries themselves, using SQL or other tools.
 - Example: Data analysts extracting trends from sales or student performance data.
3. **System Analysts**
 - Analyze user requirements and design **efficient applications** that use the database.
 - Ensure that application logic interacts correctly with the database schema.
4. **Database Designers**
 - Decide the **logical and physical structure** of the database.
 - Define **tables, relationships, constraints, keys**, and normalization rules.
5. **Database Administrator (DBA)**
 - Controls and manages the overall database system.
 - Provides **security, backup, recovery, and performance management**.
6. **Other Users (Optional/Modern DBMS)**
 - **Tool Users:** Use reporting or visualization tools connected to the database.
 - **Operators:** Monitor database servers, manage storage, and ensure uptime.

In short:

- Users include **programmers, end users, analysts, designers, DBA**, and sometimes operators or tool users.
- Each user type has **specific responsibilities**, from developing applications to maintaining security and performance.

Q5. Who is a DBA (Database Administrator)? What are their functions? (Expanded)**Answer:**

- A **Database Administrator (DBA)** is a professional responsible for **managing, securing, and maintaining the database system** to ensure smooth and efficient operation.
- The DBA acts as a **manager of the database environment**, ensuring data is safe, consistent, and accessible.

Functions of a DBA:

1. **Database Design**
 - Defines the **logical and physical schema** of databases.
 - Ensures tables, relationships, and constraints are designed for efficiency.
2. **Access Control**
 - Provides **authentication and authorization** to users.
 - Ensures users can only access data they are allowed to.
3. **Data Security**
 - Protects the database from **unauthorized access or attacks**.
 - Implements **encryption, roles, and security policies**.
4. **Backup & Recovery**
 - Plans and performs **regular backups**.
 - Restores data in case of system failures, crashes, or accidental deletion.
5. **Performance Tuning**
 - Monitors query performance and optimizes database operations.
 - Creates indexes, tunes queries, and manages storage for faster access.
6. **Data Integrity & Consistency**
 - Ensures that data follows **constraints and rules** (primary key, foreign key, check constraints).
 - Prevents anomalies and maintains correctness of data.
7. **User Management**
 - Creates new users and assigns privileges.
 - Manages roles, groups, and access levels.
8. **Monitoring & Maintenance**
 - Monitors **server performance, storage utilization, and logs**.
 - Performs routine maintenance to avoid downtime.
9. **Troubleshooting & Support**
 - Investigates **errors, crashes, and performance issues**.
 - Provides support to developers and end-users for database-related problems.

In short:

- The DBA is the **guardian and manager** of the database.
 - Responsibilities include **design, security, backup, recovery, performance tuning, data integrity, user management, and monitoring**.
 - Without a DBA, a large database system cannot run efficiently or securely.
-

Unit 2: Relational Model (10%)

Q1. Define relation, tuple, attribute, domain with examples.

Answer:

1. Relation:

- A relation is essentially a **table in a database**.
- It consists of **rows (tuples)** and **columns (attributes)**.
- Represents a collection of similar **entities** or objects.
- **Example:**

Table: Student

Roll_No	Name	Branch	CGPA
101	Rahul	CSE	3.8
102	Amit	ECE	3.6

2. Tuple:

- A tuple is a **single row** of the table.
- Represents a **single record** in a relation.
- **Example:** (101, 'Rahul', 'CSE', 3.8) → One student's record.

3. Attribute:

- An attribute is a **column** in a table that represents a **property of the entity**.
- **Example:** Roll_No, Name, Branch, CGPA are attributes of Student.

4. Domain:

- Domain is the **set of possible values** that an attribute can take.
- **Example:**
 - Roll_No → Positive integers
 - Name → Strings of length ≤ 50
 - CGPA → Decimal values from 0.0 to 10.0

In short:

- Relation = Table
- Tuple = Row
- Attribute = Column
- Domain = Allowed values of an attribute

Extra points for 6–7 marks:

- Every relation must have a **unique name**.
- Tuples in a relation must be **unique** (no duplicates).
- Domains ensure **data integrity** by restricting invalid values.

Q2. Differentiate between schema and instance of a database.

Answer:

• Database Schema:

- The **design or blueprint** of the database.
- Defines **tables, attributes, relationships, and constraints**.
- Permanent and rarely changes.
- **Example:**
Student(Roll_No, Name, Branch, CGPA) → Defines table and attributes.

• Database Instance:

- The **actual data stored** in the database at a particular time.
- Temporary; changes frequently with insert, update, delete operations.

- **Example:**
- (101, 'Rahul', 'CSE', 3.8)
- (102, 'Amit', 'ECE', 3.6)

Aspect	Database Schema	Database Instance
Definition	Structure of database – tables, attributes, relationships	Actual data stored at a specific time
Nature	Permanent	Temporary (changes frequently)
Example	Student(Roll_No, Name, Branch, CGPA)	(101, 'Rahul', 'CSE', 3.8)
Change	Rarely changes	Changes frequently (insert/update/delete)
Purpose	Guides design & integrity	Shows current state of database

- Schema ensures **data consistency** and integrity.
- Instance is used by users and applications to **retrieve, insert, update, and delete data**.
- A schema can have multiple instances over time.

In short:

- Schema = Database design/structure
- Instance = Current content/data of the database

Q3. Explain different types of keys: superkey, candidate key, primary key, foreign key, alternate key. IMP

Answer:

1. **Superkey:**
 - A set of one or more attributes that can **uniquely identify each tuple** in a relation.
 - **Example:** {Roll_No}, {Roll_No, Name} → Both can uniquely identify a student.
 - Superkeys may have **extra attributes** not necessary for uniqueness.
2. **Candidate Key:**
 - A **minimal superkey** – no attribute can be removed without losing uniqueness.
 - **Example:** {Roll_No} → Minimal set to uniquely identify a student.
3. **Primary Key:**
 - A **chosen candidate key** to uniquely identify tuples. Cannot contain NULL values.
 - **Example:** Roll_No is chosen as primary key for Student table.
 - **Purpose:** Ensures **uniqueness and integrity** of each record.
4. **Foreign Key:**
 - An attribute in one table that **references the primary key** of another table.
 - **Example:** Dept_ID in Student table refers to Dept_ID in Department table.
 - Ensures **referential integrity**, i.e., linked data exists in the referenced table.
5. **Alternate Key:**
 - Candidate keys **not chosen as the primary key**.
 - **Example:** If Roll_No is primary key, Email or Phone_No could be alternate keys.

Extra points for 6–7 marks:

- Keys ensure **uniqueness of records**.
- They are used for **efficient searching and joining tables**.
- Helps maintain **data integrity** across relations.

In short:

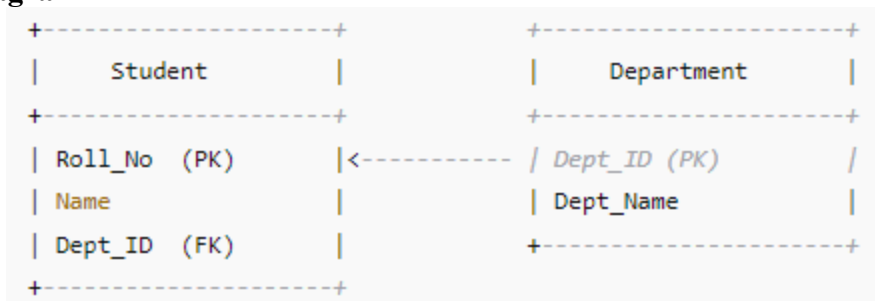
- Superkey → Any unique set of attributes
- Candidate key → Minimal unique set
- Primary key → Chosen candidate key
- Foreign key → References another table
- Alternate key → Candidate key not chosen as primary

Q4. Draw and explain a schema diagram with an example.**Answer:**

- A **schema diagram** is a **graphical representation of a database schema**.
- It shows:
 1. **Tables (relations)**
 2. **Attributes (columns)**
 3. **Relationships** between tables
 4. **Primary and foreign keys**
- **Purpose:**
 - Helps **visualize the database structure**
 - Makes it easier to design, understand, and query the database
 - Helps ensure **data integrity and proper relationships**

Example Scenario: Student and Department

1. **Tables:**
 - Student(Roll_No, Name, Dept_ID)
 - Department(Dept_ID, Dept_Name)
2. **Relationship:**
 - Each student **belongs to one department**
 - Dept_ID in Student is a **foreign key** referencing Dept_ID in Department
3. **Keys:**
 - Student → Roll_No is **primary key**, Dept_ID is **foreign key**
 - Department → Dept_ID is **primary key**

Schema Diagram**Explanation:**

- **Primary Key (PK):** Uniquely identifies each record in a table.
 - Roll_No in Student table
 - Dept_ID in Department table

- **Foreign Key (FK):** Links one table to another, ensuring **referential integrity**.
 - Dept_ID in Student table refers to Dept_ID in Department table
- **Relationship:** One-to-many (1:N) → One department can have many students, but each student belongs to only one department.

In short:

Schema diagram visually shows **tables, attributes, keys, and relationships** in the database.

Q5. List and explain relational algebra operators with examples. (IMP)**Answer:**

Relational Algebra = Set of operations on **relations** to get new relations.

1. Select (σ)

- Selects rows based on a condition.
- Example: $\sigma_{\text{Branch}='CSE'}(\text{Student}) \rightarrow$ all CSE students

2. Project (π)

- Selects columns (attributes) from a table.
- Example: $\pi_{\text{Name, Roll_No}}(\text{Student}) \rightarrow$ only Name and Roll_No

3. Union ($\cup$)

- Combines tuples from two relations (duplicates removed).
- Example: $\text{Student} \cup \text{Alumni} \rightarrow$ all people in Student or Alumni

4. Set Difference ($-$)

- Tuples in one relation but not in another.
- Example: $\text{Student} - \text{Alumni} \rightarrow$ students who are not alumni

5. Cartesian Product ($\times$)

- Combines tuples from two relations.
- Example: $\text{Student} \times \text{Department} \rightarrow$ every student with every department

6. Rename (ρ)

- Renames a relation or attributes.
- Example: $\rho_{\text{NewStudent}}(\text{Student}) \rightarrow$ renames Student as NewStudent

7. Join ($\bowtie$)

- Combines tuples based on a condition.
- Example: $\text{Student} \bowtie_{\text{Student.Dept_ID}=\text{Department.Dept_ID}} \text{Department} \rightarrow$ student info with department name

8. Intersection ($\cap$)

- Tuples common to both relations.
- Example: $\text{Student} \cap \text{Alumni} \rightarrow$ students who are alumni

9. Division ($\div$)

- Useful for “for all” queries.
- Example: Find students who took **all courses** in a department

In short:

- Relational algebra operators help **manipulate tables** to retrieve desired data.
- Core operators: **Select, Project, Union, Difference, Cartesian Product, Join, Division.**